Tetris.js audit blueprint instinctVersion1.0

Safe by construction, small trusted core

Status: Audit blueprint, added to the repository by pull request 43. AUDIT.md records its checksum in D9 ; the checksum of the text as first amended is in D10 (historical), the checksum of the text Phase 0 ran under is in D11 (historical), and D26 records the historical amended source identity. The latest publication-fidelity table in AUDIT.md identifies the current sanitized text and PDF bytes. The owner's approval of this text is confirmed separately from this line and is not asserted here. Phase 0 was executed under the D11 text and closed under D13 (pull request 46); B1 points to its record. The owner approved the Phase 1 requirement scope on October 4 (pull request 47); AUDIT.md records it in D15. AUDIT.md also records the Phase 1 state categories and authoritative transition boundary in D16, the additive amendment to D16 in D17, and the R1 decision in D18. AUDIT.md records the R2 to R5 decisions and the additive D16 Amendment 2 in D20 and D21, the SAF-3 fault reporting and timer trust decisions in D22, the storage, SAF-5 and SAF-4 timer scope decisions in D23, the trust assumptions in D24, and the property mapping and conflict register in D25. No Phase 1 exit has been approved or recorded in AUDIT.md. A Phase 1 decision takes effect on main only when AUDIT.md records it with the owner's approval. Process, evidence vocabulary, transition-boundary framework, persistence-ordering framework, research, baseline, approval, falsification, and test-minimization rules are locked when the owner approves this version. R1-R5 are explicit Phase 1 requirement/security decisions and are not unresolved blueprint mechanics. The pre-audit repository verification in 1.1 is not Phase 0. No code change, proof-tool change, repair, architecture change, or Swift rewrite is assumed or authorized. This version corrects factual and internal-consistency errors found in v5.11, restores rules agreed in earlier versions that v5.11 omitted, and includes revisions from the owner's review messages. Its changes are listed in the Changelog.

Supersedes: every earlier version and draft, including claudeVersion1.0 to claudeVersion1.4, the earlier instinctVersion1.0 files, v5.11, the copy of it labeled v5.12, and chatgptVersion1.0. This version combines them. The name instinctVersion1.0 is kept for the latest blueprint.

Changelog. Drafted by the assistant. It records what changed and where each change came from; it is not evidence that the owner approved any change. The owner approves the blueprint as a whole. Nothing here has been run, and no code was changed.
* Base: the supplied v5.11 text (chatgptVersion1.0), then claudeVersion1.0 (the same text with repository corrections), then the instinctVersion1.0 working drafts, then claudeVersion1.1 to claudeVersion1.4, as supplied by the owner with a decision record and step diffs. The decision record was written by a separate session; its quotations of owner answers are not independently authenticated here.
* Carried from claudeVersion1.4 and the owner's statement in the final review that four positions stand: the safe terminal state is defined from SAF-3's own text; N1 is a separate record cross-referenced to F20; only a claim certified Proven closes an unexhausted falsification domain; the shipped score check is loosened only as REQUIREMENTS.md rule 2 allows. The same lineage supplies proof by exhaustive enumeration with its 5.2 status, "same outcome" in test removal, L1's evidence-gap framing, F22's WebIDL conversion, approval for stopping-rule limitations, scheduler trust in the Phase 1 exit, code formatting, and the Appendix C factual fixes.
* Added after the claudeVersion1.4 base, from the owner's review messages in that review round and drafted by the assistant: R2 covers game-over forgetting (STA-3) and does not assume "forget" means deletion; the good-save definition is independent of the current validation sequence and counts saves already in storage at start; Phase 1 defines transitions as specified legal operations and decides commit-only versus intermediate-state validity; the Phase 1 requirement-scope table; F20 reproduction classes and the F23 folding rule; Phase 3 mapping appropriate to the claim and covering adopted properties; Phase 5 cannot close with a Candidate finding or a Not Assessed property; ledger, Score and ID-list fixes; survey attribution in Appendix C.
* Added in the fourth review round (assistant drafting; the owner approves the blueprint as a whole): R4 decides whether a save written during a step that then faults counts as the last good save, and the good-save content definition does not settle it; F20 has a Correspondence question; "reported" is stated as an owner interpretation because REQUIREMENTS.md gives no method or audience (checked: only SAF-3 and SAF-5 use the word); Phase 1 fills only Scope and Owner decision in the scope table.
* Added in the third review round (assistant drafting; the owner approves the blueprint as a whole): the Apple statement now says O1 to O8 are the owner's adopted properties with Apple material as inspiration only and no cited source; the scope-table rows are labelled illustrative; F20 has an applicability question; Phase 1 exit has an observable "fault reported" criterion; Phase 2 records a reason for each excluded file.
* Added in the previous review round (assistant drafting; the owner approves the blueprint as a whole): "forgetting" replaces "deletion" in F20, 4.7 and N1, and the N1 record is corrected against gameManager.js:194; the good-save definition separates persisted-state validity from transition ordering; Phase 4.1 states that the implementation predicate cannot supply its own expected result; F20 separates the ordering question from the consequence question and states that the high-score write is not presumed to follow the saved-game rule; Phase 1 defines "reported" independently of the callback; a concrete requirement-scope table template; the status wording separates the persistence-ordering framework from the open R2 and R4 semantics. The three earlier history blocks were removed as superseded by this Changelog, and the "(drafted)" marker convention was dropped.
* Amendment D10 (assistant drafting, for owner review; not owner approval): the Status line gives checksum locations and does not assert approval; B2 names the service worker's `cache.addAll` network call; Phase 0 points to B1 as the single list, and B1 adds the mutation threshold and repository configuration facts and keeps its earlier wording for mutant outcomes and deployment result; 1.2 gains two sentences saying the repository holds assistant-written secondary summaries of Apple's documentation (Appendix D) and that they do not change the statement; the first line of Appendix A says no source is cited and the summaries are not cited as grounding; the Appendix C source line gives the survey's path in docs/sources; Appendix D lists the kept source files, says which are relied on, and records the Google summary's provenance and the October 4 check against Google's official PDF.
* Amendment D11 (assistant drafting, for owner review; not owner approval): O8 gains an authorship-independence rule (a model input pack prepared by someone else and approved by the owner; production code, production tests, and implementation-derived fixtures and expected outputs barred during construction, including saved-game fixtures from released versions; the freeze point; post-freeze changes need a new checksum and a requirements-based justification); 3.1 refers to it; 4.8 adds two cache attacks, foreign `tetris-version-` cache serving and unnumbered `tetris-version-` names, as attacks to run and not as new requirements; Phase 4 runs missing 4.8 attacks as scratch tests and records any apparent APP conflict as a Candidate finding; the Status line distinguishes the historical D10 checksum from the current D11 checksum.
* Amendment D26 (assistant drafting; the owner approved the October4 proposal on October5,2026, not the final text or checksum): record hygiene only. The Status line and the reservation sentence now match D20 to D25 as recorded in AUDIT.md, a status line under each of the R2, R3, R4 and R5 headings points to D21 as the R1 heading does to D18, and the Appendix E introduction no longer calls the Phase 1 trust-assumption decisions open. D26 retains the historical source identity; the latest AUDIT publication-fidelity record identifies the current public text. This records no new decision. D19 to D25, REQUIREMENTS.md and production code are unchanged, and no Phase 1 exit is claimed.
* Amendment D19 (assistant drafting; finalized under the owner's October4,2026 docs-only delegation to merge after CI and report completion; final text and checksum were not shown or approved): Appendix A cites sources for the themes they support and says which themes have no verified source; Appendix C6 gains source-type labels and no link is replaced; Appendix E records point-in-time trust-ledger candidates and non-goals; the Status line gives D19 as the location of the current text's checksum and records the effect of D15, D17 and D18. The Status line, the reservation

sentence and the R1 heading are updated to match D15 to D18 as already recorded in AUDIT.md; this is record hygiene and records no new decision. No requirement, control, test or semantic decision is added, and D17, D18 and R1 are not changed.
* Amendment D14 (assistant drafting, for owner review; not owner approval): the Status line records that Phase 0 ran under the D11 text and closed under D13, and that the owner-approved Phase 1 requirement scope is held on an unmerged pull request and is not yet recorded on main, with no other Phase 1 decision approved; B1 points to the approved Phase 0 record and its checksum; 1.1 says that "in scope" is not certification and that unavailable device or browser evidence is an evidence gap, not an automatic exclusion; 1.3 and O1 preserve SAF-4's literal wording when the transition boundary is defined; Phase 1 requires its decision text to be normative and separate from implementation evidence, for the O8 model input pack; Phase 1 records the sequence of the scope table before the state-model and boundary proposal, as separate documentation-only changes. This amendment does not change Phase 0's exit standard retroactively and does not invalidate D13.

Reserved to the owner: any change to REQUIREMENTS.md or to the D8 behavior, approval of this blueprint, and every phase exit. R1 was decided in D18 and R2 to R5 in D21; they are no longer open.

Scope: soliskit/tetris-js

Audited main commit: 30e0085

Purpose: Establish a rigorous, independently grounded correctness and safety case for the existing JavaScript Tetris engine and page layer, identify only concrete requirement violations or evidence gaps, make the smallest justified production changes, and finish with a minimized trusted core and permanent regression suite.

—

Governing rule

The audit is performed in this order:

audit → findings → classification → decision → approved change → implementation → re-prove/re-falsify → certification → minimization

Audit only mechanisms and requirements that actually exist or are explicitly adopted. Do not invent requirements, strengthen invariants, or classify theoretical weaknesses as defects without evidence.

Findings belong to the requirement they violate, even if discovered while auditing another mechanism.

No production code changes occur during investigation.

No implementation is performed merely because a candidate defect looks plausible.

No requirement is strengthened merely because a stronger invariant would be mathematically convenient.

No existing protection or test is weakened merely to reduce code or test count.

Every confirmed production defect receives a regression test that fails before the correction and passes after it.

Every final certification claim identifies its evidence method, assumptions, covered code or model, and limitations.

—

Audit Advancement Gate

A phase may advance only when its preceding phase's exit criteria are explicitly recorded and approved by the user/project owner.

Before implementing any production change, architecture change, requirement change, or permanent-test removal, the audit record must contain:

1. the applicable requirement or adopted security property;
2. the authoritative transition boundary and affected production routes;
3. the applicable finding status, separately from property-certification status;
4. the proof method or testing method proposed for the claim;
5. the assumptions, covered code/model, evidence required, and known limitations;
6. the declared falsification procedure, partitions, and stopping bound, where applicable;
7. the disposition of every identified candidate finding, including Not Reproduced where appropriate;
8. the required requirement/security-scope decisions;
9. the proposed correction and its regression test, for confirmed defects;
10. explicit user/project-owner approval for any decision that changes requirements, security scope, production behavior, architecture, or the permanent test set.

No-counterexample results, successful tests, reviewed arguments, and machine-checked proofs are not interchangeable forms of evidence.

A phase is not complete merely because its work was attempted. It is complete only when its declared evidence and exit criteria are satisfied, or an explicit Not Established or Out of Scope disposition has been approved.

Requirement or security-scope decisions may be proposed during the phase responsible for resolving them, but may not be implemented without explicit user/project-owner approval.

The gate controls advancement and authorization. It does not require Phase 1 questions to be answered before Phase 1 begins.

Responsibility separation and parallel preparation

Independent evidence gathering, correction preparation, independent review and final integration may proceed concurrently when their information and artifact dependencies are satisfied. Concurrent preparation does not close a phase or authorize implementation.

Each work item records its claim or correction scope, responsible owner, input versions, proposed output artifacts, read/write set, upstream dependencies, independent reviewer and remaining approval or exit gates. Assign ownership by artifact or bounded claim rather than treating an entire phase as a single indivisible work item.

Responsibilities are separate:

* Evidence gathering binds requirements, source and evidence identities, covered domains, assumptions, limitations and candidate dispositions. It does not supply its own independent review or certify a property merely by collecting results.
* Correction preparation proposes a minimal conceptual change, a regression with independently justified expected results, and the protections plausibly affected. Preparation is not production or permanent-test implementation permission.
* Independent review evaluates the named immutable artifact and its evidence at an exact version. A content review and final exact-version verification are separate gates. Changed inputs or outputs require reassessment of the affected review; an earlier pass is not transferred by assumption.
* Final integration preserves governing records, sequences shared writes, binds the final reviewed versions and records verification and publication results. Integration cannot invent requirement choices, change evidence scope or exercise approval outside the applicable owner grant.

Independent artifacts may be prepared in parallel. Shared governing records, approved model-input packs and overlapping source files have one writer at a time. Conflicting changes are sequenced or reconciled before final review. Each work item identifies cross-cutting dependencies, including fault cleanup, reporting, persistence ordering and timer/input lifecycle where relevant; responsibility separation does not remove those dependencies.

O8's model-input preparation, owner approval, pre-freeze authorship isolation, checksum freeze and independently justified later model changes remain unchanged. A responsibility assignment does not permit production behavior, tests or implementation-derived expectations to enter pre-freeze model construction.

The Audit Advancement Gate, numbered phase-exit criteria, mandatory candidate reproduction order unless separately changed, finding/property distinction, production and permanent-test approval, pre-fix failing/post-fix passing regression, affected-protection re-verification and final certification remain unchanged. A blocked work item does not block unrelated authorized preparation. Per-defect implementation before the applicable phase gates would require a separate explicit amendment; it is not authorized by this section.

___

Evidence vocabulary

Proof Method

The method by which a universal claim is established.

A final claim must identify the actual method used.

Tested

Observed to satisfy the property for the specified test inputs, state classes, and bounds.

Testing does not establish behavior outside the tested domain unless another argument covers that domain.

Proven

Proven means established by a stated deductive proof method over a stated domain under stated assumptions: a reviewed deductive argument, a machine-checked proof, or exhaustive enumeration of the claim's complete finite domain against an independently justified oracle. Any other testing produces a Tested claim, not a Proven one.

The method must be named.

A reviewed deductive argument and a machine-checked formal proof are different methods; only the latter is Formally Verified.

Formally Verified

Reserved for an actual machine-checked formal proof.

A reviewed argument is not formally verified merely because it is detailed or rigorous.

Not Established

Available evidence is insufficient to make either a proof or testing claim at the required scope.

Not Reproduced

A specified reproduction attempt did not reproduce the reported behavior.

Not Reproduced is not equivalent to Rejected and does not by itself establish correctness.

Rejected

The concern was evaluated and does not violate the applicable requirement or adopted property.

Evidence Gap

Required evidence for a finding or property claim has not yet been obtained. It does not determine finding status or property-certification status.

Evidence gaps are tracked independently from finding status.

___

0. Baseline

Phase 0 establishes the audit starting point.

The values listed in Appendix B1 must be recorded from the actual repository and CI before Phase 0 closes. B1 is the single list.

Existing observations from 30e0085 are baseline evidence only. They are not independently verified until reproduced where this blueprint requires reproduction.

PR numbers are historical context. They are not proof merely because they are recorded in the repository history.

Phase 0 exit

Phase 0 closes only when the required Appendix B values, including the CI result, are actually recorded.

—

1. Security and correctness model

1.1 Requirements

The authoritative project requirements are defined in REQUIREMENTS.md.

For the requirements that govern the core safety and correctness proof, this blueprint reproduces the exact wording from REQUIREMENTS.md at commit 30e0085. The remaining requirements are cited by ID and remain governed by REQUIREMENTS.md. IDs cited elsewhere in this blueprint without reproduction (including STA-3, STA-6, PCE-1 to PCE-6, APP-2, APP-3, SCO-1, SCO-2, INP-1, INP-2, PLY-3 and PLY-6) were checked to exist in REQUIREMENTS.md at 30e0085 during this revision; only the IDs reproduced here are quoted.

Applicable requirements are all REQUIREMENTS.md requirements within the declared audit scope, plus any security properties explicitly adopted by owner approval. Scope is declared in Phase 1 as an explicit requirement-scope table: every REQUIREMENTS.md ID is marked In Scope, Out of Scope, or Not Yet Scoped, and no requirement may be excluded merely because it is inconvenient to prove. A requirement outside scope is listed as Not Certified, not silently omitted.

"In Scope" means the audit undertakes to address and certify the requirement if sufficient evidence can be obtained. It does not mean the requirement is established or certified. Evidence that cannot be obtained, for example because it needs a particular device or browser, is recorded as an evidence gap against the requirement. It is not by itself a reason to exclude the requirement from scope.

Pre-audit repository verification record (this revision, not Phase 0): a script compared each of the nine reproduced requirement texts (SAF-1, SAF-2, SAF-3, SAF-4, PLY-2, PLY-7, STA-1, STA-4, APP-6) against the REQUIREMENTS.md row at 30e0085, and all nine matched exactly. The same check was run on the STA-6, PCE-* and APP-* IDs for existence. This is a pre-audit drafting-time check. It is not Phase 0 and not audit evidence; Phase 0 re-runs it and records the output.

The audit must use the repository's exact requirement wording when determining whether behavior is required, permitted, rejected, or unspecified.

SAF-1

A saved game is untrusted input. Every field is checked and pieces are rebuilt from the built in definitions. Any invalid save is rejected whole and forgotten, never partly loaded.

SAF-2

Storage that is missing, blocked, full, or throws on reads or writes never stops the game. Invalid stored values are ignored.

SAF-3

If anything in the engine throws, the game stops in a safe state (game over, no timers running), the fault is reported, and the last good save is kept.

SAF-4

After every action and every timer the engine checks its invariants: board 20 by 10, three pieces queued, a valid score, and while playing the piece fits, gravity is running and the lock delay runs exactly while the piece rests, otherwise no timers run. A broken invariant is handled like SAF-3. Legal play never breaks one.

PLY-2

Gravity moves the piece down one row every max(0.25, 0.7 - 0.02 × (level - 1)) seconds while playing.

PLY-7

Hold puts the current piece aside in its starting state and brings in the held piece, or the next piece if none is held. Hold works once per piece.

STA-1

The game opens at game over. New Game works from game over, or while paused to give up that game, never while playing. It resets the board, score, level, held piece, queue and bag, and forgets the saved game. While paused it first asks in a dialog ("Start a new game? This game will be lost."), so one stray press cannot end a game: New Game in the dialog gives the game up, while Cancel, Escape, a click outside the dialog, or any other action before the answer keeps it paused; a click inside the dialog, away from its buttons, does nothing. Cancel is selected when it opens, so a second Enter keeps the game, and the selected button is always highlighted. While the dialog is open, keys go to its buttons, the left and right arrow keys move between them, and a held key repeating presses nothing.

STA-4

Pausing saves the game, and so does clearing lines. Continue, from game over only, restores exactly what was saved, paused, including the pieces left in the bag. A save from before the bag was saved continues with a fresh bag.

APP-6

GitHub Pages serves every project of an account from one origin, which they all share storage and caches with, so the game touches only its own: it stores the high score and saved game under names of its own, and deletes only caches it made.

The following remain requirement-controlled and are cited by ID:

* SAF-5
* SAF-6
* INP-3
* INP-4
* QA-1
* QA-2
* QA-3
* QA-4
* QA-5
* QA-6

No audit conclusion may silently replace a requirement with a stronger or weaker rule.

—

1.2 Adopted security properties

The properties O1 to O8 below are security properties adopted by the project owner for this audit. Apple's published security architecture was design inspiration for the kind of property chosen. This blueprint cites no Apple source and does not claim that O1 to O8 are derived from, or conform to, any Apple document; no source evidence for such a derivation is held here. If the owner wants a specific Apple property cited as grounding, the source and the property it supports must be added to Appendix A first. Platform-specific mechanisms that depend on hardware, OS isolation, cryptography or other facilities absent from this JavaScript project are not adopted. Assistant-written summaries of Apple's published security documentation are kept in the repository (Appendix D). They are secondary sources, not Apple's own pages, and they do not change this statement.

O1. Authoritative transition boundary

A transition is a specified legal operation that attempts to move the authoritative state from one certified state to another certified state, or to the certified safe terminal/fault state. Phase 1 independently defines which states are valid, which operations are legal, and what each operation is permitted to change.

The Phase 1 model must also define whether a transition is evaluated only at its commit point or whether intermediate mutations must themselves preserve validity. Where intermediate mutations are permitted, the model defines the failure or rollback semantics that make a thrown transition safe. The audit assumes neither atomicity nor a rollback mandate unless the requirements provide one.

Defining the boundary does not narrow when SAF-4 requires invariants to be checked. SAF-4 requires the engine to check its invariants "after every action and every timer", and the model preserves that wording literally. Reading it as a check at fewer points, for example only at commit points, is a requirement clarification or change that needs the owner's approval. It is not a modelling choice.

A certified authoritative state is a state satisfying the Phase 1 valid-state model (1.3), or the safe terminal state as defined in 1.3. Phase 2 uses this definition and does not redefine it while inventorying routes. Until Phase 1 approval, this definition is a specification target, not a production predicate and not evidence of correctness; it is never defined retrospectively from what the implementation accepts. After Phase 1 approval it is the approved audit definition, and production code is judged against it.

The authoritative transition boundary is the conceptual commit boundary at which a specified legal operation commits the changes Phase 1 permits it to make to authoritative state, or enters the certified safe terminal/fault state, including authorized persistence effects belonging to that transition.

The boundary is defined independently of the current implementation. Phase 2 maps it onto the production code and inventories every route that can reach or bypass it.

Existing functions do not become authoritative merely because they contain validation.

Any privileged exception is explicit and separately proved.

Whether multiple implementation routes should be consolidated is a Phase 6 construction question, not a premise of the audit.

O2. Valid transitions only

Every legal engine transition must preserve the defined valid-state predicate or intentionally enter a defined safe terminal state.

Malformed external state must not become a legal engine state merely because a representation happens to satisfy a shallow predicate.

O3. Validated persisted state

Persisted state is untrusted input.

A persisted game may enter authoritative engine state only through complete validation and reconstruction from trusted built-in definitions.

Invalid persisted state is rejected as a whole.

O4. Persistence ordering

For persistence effects where the applicable requirements require preservation of the last-known-good persisted representation, the transition must not destroy that representation before the corresponding engine transition has reached the required post-transition validation boundary.

This ordering rule applies only where preservation is required. The authority order is: REQUIREMENTS.md, then an owner-approved requirement clarification or change, then adopted security properties such as O4, then implementation interpretation. O4 is subordinate to the applicable requirement semantics. An owner decision on R2 or R4 cannot silently override an existing requirement: if a decision would contradict a requirement (for example SAF-3 or STA-1), the requirement must first be clarified or changed through the owner-approved route, and until then the conflict is recorded and not certified. O4 does not by itself decide those questions.

For this audit, “last known-good” means the persisted representation observable through the application's storage interface. Stronger crash/power-loss durability is outside the certification claim unless independently established.

What makes a persisted representation "good" is defined from the requirements and the Phase 1 valid-state model (1.3), not from the production parser or the implementation's current validation sequence. A newly written save is good when it encodes a state that is valid under the Phase 1

model and satisfies the applicable persisted-state requirements (SAF-1 and STA-4). A save already in storage when the game started is good when it encodes such a valid persisted state (SAF-1, STA-4 and STA-6; STA-6 requires supported earlier formats to keep loading, and such a save cannot be known to have passed the current implementation's checks), regardless of whether the current implementation would accept it. This is the default content definition. It does not mention the transition that wrote the save, so it does not depend on what O4 and F20 examine, and it does not decide whether a save written during a step that then faults counts as the last good save; that is decided under R4. O4 then asks separately whether a transition preserves that last good representation until the required post-transition validation boundary. Validity of the persisted state, correspondence of what is persisted to the committed state the requirement calls for (examined in F20), and implementation ordering are three separate questions. Whether `parseSession` accepts a stored value is evidence about that definition, not the definition. A disagreement between the model-side validity predicate and `parseSession` is itself a candidate finding.

This definition covers the saved game only. What makes the saved-game flag and the stored high score good is decided under R4.

A required forgetting effect is not automatically treated as a replacement write.

R2 must explicitly determine:

* what persistence effect New Game (STA-1) and game over (STA-3) each require, and whether "forgotten" means deletion;
* exactly when that effect commits;
* whether a replacement persistent state exists;
* what must survive if New Game, or the step that ends the game, fails after or around that effect;
* whether the existing REQUIREMENTS.md text already determines the result, or an explicit requirement clarification or change is required first.

O5. Code-to-claim link

Every implementation claim identifies exactly which shipped code establishes the claim and what evidence establishes that link.

A statement that a model or test covers a property does not by itself establish that the shipped implementation corresponds to that model or test.

O6. Trust ledger

Every trusted component is explicitly recorded with:

* its responsibility;
* why it is trusted;
* the evidence supporting that trust;
* assumptions placed on it;
* known failure modes;
* whether it is inside or outside the audited proof boundary.

O7. Boundary enforcement

The authoritative transition boundary must actually constrain production routes.

Any deliberate bypass must be explicit, narrow, documented, and separately proved.

A route that can mutate authoritative state without satisfying the boundary is a candidate finding. It is a defect only if it violates a REQUIREMENTS.md requirement (for example SAF-3 or SAF-4) or an owner-adopted requirement, unless explicitly authorized by the requirements and trust model. Its regression test is tagged with that requirement's ID, because test/traceability.test.js fails any test that names an ID absent from REQUIREMENTS.md. Adding the O properties to REQUIREMENTS.md as rows would be a requirement change needing the owner's approval; this blueprint does not assume it.

O8. Independent executable reference model and correspondence

The reference model is independently constructed and executable.

It must not mechanically reuse implementation transition logic or validation helpers.

The model and production implementation must not share faulty logic capable of making both agree on the same incorrect behavior.

Correspondence between the model and implementation is established through independently designed differential tests and an explicit independence argument.

Correspondence evidence is not treated as proof merely because both sides agree.

Authorship independence: this limits how the model is constructed. The model author builds the model only from a model input pack. Someone other than the author prepares the pack, the owner approves it, and its contents are recorded by checksum. The pack contains only: (1) REQUIREMENTS.md; (2) the text of the owner's approved Phase 1 decisions, including the applicable state categories and transition/property scope, with implementation names, code locations, test names, observed behavior and finding records removed, except that an identifier of a persisted or user-visible field stays where the decision cannot be stated without it; (3) a published specification or oracle that the owner adopted in Phase 1, as published. The author does not receive the rest of this blueprint or AUDIT.md, because they report implementation behavior, and must not read the production implementation code, the production tests (test/ and e2e/), or fixtures and expected outputs derived from the implementation, including saved-game fixtures from released versions. Where this cannot be met, the exception, the reason and the independence argument that replaces it are recorded. The owner must approve the exception before model construction proceeds. An exception cannot permit exposure to production behavior unless the owner accepts the replacement independence argument. The limit ends when the model is frozen: its source and the pack are recorded by checksum before any comparison with the implementation. After the freeze the author may take part in differential testing and correspondence review. After the freeze any change to the model gets a new checksum and a written justification from REQUIREMENTS.md or an approved Phase 1 decision that does not rely on the implementation's behavior; a change made only to agree with the implementation is recorded as a correspondence result, not as a model fix.

—

1.3 Exact meaning of valid state

The audit defines validity from requirements and adopted properties before judging whether current predicates are sufficient.

The model first partitions state into five categories and states, for each, which transitions may commit it:

* authoritative game state;
* derived state (for example anything computable from other state, such as level if it is derived from score, or a fit predicate);
* runtime resources (for example timer handles);
* persisted representation (external state);
* presentation state (for example rendering caches), which is not authoritative engine state.

Differential comparisons under 3.4 compare model and production state within the same category. The partition is a Phase 1 decision; the audit does not assume which category an item belongs to.

The safe terminal/fault state is defined in Phase 1 from SAF-3's own text: game over, no timers running, the fault reported, and the last good save kept. REQUIREMENTS.md says only that the fault is "reported" (SAF-3 and SAF-5); no requirement says how or to whom. Phase 1 decides what counts as reported, with the owner's approval and independently of whatever callback the implementation currently invokes (including `onFault`); otherwise O5 would be circular. This is an interpretation for the owner, like R1, and not a requirement the audit may assume. The definition adds no condition on the board, queue, current piece, score or hold that SAF-3 does not state, because a fault normally leaves those contents malformed and the safe stop keeps them as they are (AUDIT.md F18). Any such condition needs an owner-approved requirement change. Because the definition is explicit, it cannot serve as a catch-all for states the valid-state model rejects.

At minimum, the model must resolve:

* board dimensions and cell domain;
* queue length and queue-element validity;
* current-piece validity;
* current-piece position and rotation;
* whether the current piece must fit in each state;
* score domain;
* level domain and score/level relationship;
* hold representation;
* `canHoldTetromino` semantics;
* bag representation and uniqueness;
* gravity state;
* lock-delay state;
* game-over/playing/paused relationships;
* persistence representation;
* which data is authoritative, including persisted values and the New Game confirmation flag (`isConfirmingNewGame`);
* any state combinations prohibited by the requirements.

A JavaScript representation that can encode an impossible combination is not automatically evidence of a defect. The question is whether the authoritative transition boundary can create or accept that combination and whether the requirements prohibit it.

1.4 Failure and attack model

The audit considers:

* malformed saved data;
* invalid persisted field combinations;
* storage read/write failures;
* serialization failures;
* engine exceptions;
* invariant-check failures;
* timer/scheduler failures where inside the trust boundary;
* drawing failures;
* gamepad-reading failures;
* malformed internal state injected at the authoritative boundary;
* legal gameplay traces;
* boundary bypasses;
* protection-removal mutations;
* circular-oracle failures;
* shared-origin storage/cache interference relevant to APP-6.

Hand-edited storage is not automatically classified as illegal gameplay.

Whether hand-edited persisted state is included in a particular gameplay requirement is an R5 decision.

1.5 Resolution questions

These are intentional Phase 1 decisions.

R1. Exact valid-state semantics

Status: decided in D18 (docs/phase-1-r1-decision.md). The question below is kept as originally written.

Resolve the exact meaning of:

* valid board;

* valid queue;
* valid current piece;
* valid timer state;
* any representation-level distinction such as sparse versus dense arrays.

R2. New Game and game-over persistence semantics and requirement reconciliation

Status: decided in D21 (docs/phase-1-r2-r5-decision.md). The question below is kept as originally written.

STA-1 (New Game) and STA-3 (game over, "the saved game is forgotten") each specify a saved-game forgetting effect. R2 determines what that effect means operationally. It does not assume that "forget" means physical deletion. Determine the required persistence effect of each (deletion, invalidation, withdrawal from discovery, replacement or another effect), when it commits, and how each interacts with SAF-3 (if the engine throws, the last good save is kept). The sequence is: first determine the meaning of STA-1, STA-3 and SAF-3 from their existing text. Establish whether their persistence semantics are compatible. If compatible, record the resulting semantics and ordering requirements. If conflicting or materially ambiguous, obtain an owner-approved requirement clarification or change before implementation or certification. R2 is not a free-standing permission and is not a mechanism for deciding what the requirements mean: the owner's decision cannot override an existing requirement without that clarification or change. The audit does not decide the outcome before Phase 1.

R3. Gravity timing tolerance

Status: decided in D21 (docs/phase-1-r2-r5-decision.md). The question below is kept as originally written.

Resolve whether PLY-2 is judged against exact mathematical interval values or an explicitly defined browser-timer tolerance.

The implementation fact and mathematical discrepancy may be established before the classification is decided.

R4. Storage failure semantics

Status: decided in D21 (docs/phase-1-r2-r5-decision.md). The question below is kept as originally written.

Resolve whether a failed storage read may make a previously discoverable save temporarily or permanently unavailable, or whether the requirement demands retry/discoverability preservation.

R4 also covers failed storage writes. AUDIT.md D8 (SAF-2) records that a failed save withdraws the previous saved game, so that an out-of-date game is never offered, with no change made. D8 is a repository record of that behavior and its reasoning. This audit has not established that the owner approved it as a requirement, and D8 is not new owner approval. R4 is the owner's decision. Until it is made, the audit records the observed behavior and classifies it neither as a defect nor as compliant on the strength of O4 alone or D8 alone. Keeping the previous save instead of withdrawing it would be a requirement change under REQUIREMENTS.md rule 1 and needs the owner's approval.

R4 also decides what makes the saved-game flag and the stored high score good for O4. F24 concerns the flag.

R4 also decides whether a save written during a step that then faults counts as the last good save under SAF-3. Both readings of "the last good save is kept" are defensible: that such a save counts if its contents are valid, or that it does not count because the step did not complete. This is an owner decision in Phase 1. The content definition of a good save in O4 does not settle it.

R5. Hand-edited persisted state and gameplay requirements

Status: decided in D21 (docs/phase-1-r2-r5-decision.md). The question below is kept as originally written.

Resolve whether deliberately modified persisted state is within the scope of gameplay requirements such as PLY-7, or only within the untrusted-input safety model.

—

Phase 1 exit criteria

Phase 1 closes only when:

* R1-R5 are explicitly decided;
* the valid-state model is recorded;
* the adopted security properties are recorded;
* the authoritative transition boundary definition is recorded;
* the trust assumptions are recorded, including whether scheduler registration and cancellation are inside or outside the trusted boundary, starting from AUDIT.md D2, F5 and F6;
* "fault reported" (SAF-3, SAF-5) has an observable criterion decided in Phase 1 with the owner's approval, because no requirement says how or to whom a fault is reported, such as a user-visible fault indication, a persistent fault record, a callback invocation, or a stated combination, recorded before O5 is certified and independent of the callback the implementation currently invokes;
* the requirement-scope table has every REQUIREMENTS.md ID in one row with Scope (In Scope or Out of Scope; none remains Not Yet Scoped) and an owner-approved decision filled; the later columns are filled in Phases 3 and 5; no requirement is Out of Scope merely because it is inconvenient to prove; every requirement outside scope is listed as Not Certified (1.1);
* the applicable requirements for each proof obligation are identified;
* any conflicting, ambiguous, or apparently inconsistent requirements have been identified and resolved by existing requirement text or by an explicit owner-approved requirement clarification or change;
* each approved Phase 1 decision is recorded as normative decision text, separate from implementation evidence: the text states the decision without code locations, test names, observed implementation behavior or finding history (an identifier of a persisted or user-visible field stays where the decision cannot be stated without it), so the owner-approved decision text can be used in the O8 model input pack without leaking implementation detail; evidence and findings are recorded in separate records;
* the user/project owner approves the resulting requirement and security-scope decisions.

Requirement-scope table template. The rows below are illustrative only; they are not the artifact. Phase 1 fills the Scope and Owner decision columns for every row. Later phases fill the rest: Phase 3 the Applicable claim and Evidence method, Phase 5 the Certification status. Each REQUIREMENTS.md ID occupies exactly one row, and no ID may be missing or duplicated:

Requirement | Scope (In Scope / Out of Scope; Not Yet Scoped only before closure) | Applicable claim | Evidence method | Owner decision | Certification status
SAF-1 | ... | ... | ... | ... | ...
SAF-2 | ... | ... | ... | ... | ...

... one row for every remaining REQUIREMENTS.md ID. Every remaining ID must occupy exactly one row, with Scope and Owner decision filled, before Phase 1 closes.

No production change occurs merely to settle R1-R5.

Sequence. The requirement-scope table is approved by the owner before the state-model and transition-boundary proposal is made, and the two are separate documentation-only changes. R1 is decided after the state categories and the transition boundary are approved. The state-model and boundary proposal does not rely on a scope row that is not yet approved.

Exit criteria for Phases 2 to 10

The phases are numbered by sections 2 to 10 of this blueprint. Each phase closes only when its criteria are recorded and approved under the Audit Advancement Gate.

* Phase 2 (inventory) closes when every route in the section 2 inventory that can affect authoritative state is mapped to the independently defined boundary, and every outer-layer route is recorded as audited for containment or bypass, each with a code reference, and every file excluded from the inventory has a recorded reason it cannot affect an in-scope claim.
* Phase 3 (correctness evidence) closes when every in-scope requirement and every adopted property has a proof-obligation record under 3.3 with a stated method and a complete mapping appropriate to the claim: requirement, applicable state property, transition class where applicable, production route or trusted component, model transition where applicable, evidence method and evidence. No artificial model or route relationship is created to fill the template. A row without the mapping appropriate to its claim does not close the phase.
* Phase 4 (falsification) closes when every declared domain is exhausted, or covered by a claim certified Proven under 5.2, or each limitation is documented, justified and approved, and every candidate has a recorded disposition.
* Phase 5 (classification and certification) closes when every finding has one finding status and every property one certification status, no finding remains Candidate, and no in-scope property remains Not Assessed.
* Phase 6 (correction) closes when every confirmed defect has an owner-approved minimal correction and a defined regression test.
* Phase 7 (implementation) closes when each approved change is made and its verification is recorded.
* Phase 8 (re-prove and re-falsify) closes when the original counterexamples are re-falsified and the evidence relevant to the corrected property is repeated, together with any protection whose behavior the correction could have affected. A persistence correction repeats the O4 evidence, a boundary correction the boundary analysis, a timer correction the timer evidence; an unrelated correction does not trigger the others.
* Phase 9 (final certification) closes when every claim names its method, assumptions, covered code or model, evidence and limitations, with owner approval.
* Phase 10 (minimization) closes when each removal states its replacement protection and the owner has approved it.

—

2. Current implementation and transition-boundary inventory

The actual engine/UI files are:

* public/game/gameManager.js
* public/game/gameState.js
* public/game/inputController.js
* public/game/position.js
* public/game/session.js
* public/game/tetromino.js
* public/game/tetrominoFactory.js
* public/script.js
* public/sw.js
* public/index.html

Other files under public/ (icons/, manifest.webmanifest, style.css) are not inventoried here, for these reasons, checked against the repository at 30e0085. icons/ holds four static image files (icon.svg and three PNG files) that contain no script and are only referenced by index.html, manifest.webmanifest and the sw.js cache list. style.css contains no script, no `@import` and no `url()` reference, so it affects presentation only. manifest.webmanifest is declarative data (name, start_url, scope, display, orientation, colours, icon references); it executes nothing and cannot change authoritative game state. Each exclusion holds only while no in-scope claim depends on the file: an in-scope presentation requirement retains style.css, and an in-scope installability or scope requirement (the APP requirements) retains manifest.webmanifest, whose start_url and scope also interact with the sw.js cache. Phase 2 confirms these reasons and records any file that fails the test.

Tests:

* test/
* e2e/

Requirements:

* REQUIREMENTS.md

The inventory must map:

* every public action entry;
* every timer callback;
* every state mutation route;
* every persistence route;
* every game-over route;
* every New Game route;
* every Continue route;

* every input route;
* every drawing callback;
* every gamepad polling route;
* every storage event route;
* every route capable of bypassing the authoritative transition boundary.

The inventory separates two kinds of entry. Routes that can affect authoritative state are mapped to the boundary. Outer-layer routes (for example drawing callbacks, gamepad polling and storage events) are audited for containment or bypass, and an entry records that it does not commit authoritative state when that is the case. Not every entry is mapped to the boundary.

The inventory must be derived from code and call relationships, not from the names of existing guard functions.

—

3. Establish the correctness evidence

3.1 Independent reference model

Construct a small executable model of:

* valid state;
* legal actions;
* state transitions;
* timer-state rules;
* persistence effects;
* safe-failure transition.

The model is independent of production transition logic and is authored under O8's authorship-independence limit on inputs.

It must not call production validators to decide whether its own expected result is correct.

The model's purpose is to make the intended rules explicit and provide an independent oracle for differential evidence.

—

3.2 Shared primitives

Shared constants or immutable domain definitions may be reused only where doing so cannot create a circular correctness argument.

Production validation logic must not automatically become model validation logic.

A shared primitive that determines the behavior under audit must be independently justified or replaced with an independent model-side representation.

—

3.3 Proof obligations

Each property must have an explicit record containing:

* requirement/property;
* the applicable state property it implies;
* the transition classes that can affect it, where applicable;
* the production routes or trusted components for those transition classes;
* the model transitions that correspond to them, where the model is applicable (no artificial model relationship is created to fill the template; where the model does not apply, the record names the actual independent evidence method);
* claim;
* Proof Method;
* assumptions;
* covered production code;
* covered model code, if applicable;
* evidence;
* limitations;
* certification status.

O5/O8 separation

O5 answers:

Why does this claim actually refer to the shipped implementation?

O8 answers:

Why is the independent model meaningful, and what evidence demonstrates correspondence without making the implementation its own oracle?

These are separate obligations.

—

3.4 Differential evidence

For each applicable transition class:

1. construct a valid model state;
2. select a legal action;
3. execute the model transition;
4. execute the corresponding production transition;
5. compare observable authoritative state;
6. compare safe-failure behavior for injected failures where applicable;
7. record discrepancies;
8. investigate whether the discrepancy is requirement, model, implementation, or mapping error.

Agreement is evidence of correspondence.

Agreement alone is not formal proof.

Appendix C lists published references (the Guideline text, the SRS kick tables, open-source engines) that could serve as independent oracles. Using one is a Phase 1 decision subject to owner approval. None is a requirement, and none shows that this implementation satisfies a property.

---

4. Falsification

4.1 State-space attacks

Attack the valid-state predicate with:

* wrong dimensions;
* sparse arrays;
* invalid cell values;
* malformed pieces;
* invalid rotations;
* invalid positions;
* malformed queue entries;
* invalid held pieces;
* invalid bag contents;
* invalid scores;
* invalid timer handles;
* impossible combinations of state flags.

Where the state space is finite and practical, enumerate it exhaustively. Exhaustive enumeration is evidence only when the enumerator and the expected valid/invalid classification are independently justified for the claimed domain. The implementation predicate under test must not supply its own expected result; the expected classification must come from the approved model, an independently justified mathematical predicate, or another independent oracle.

Where it is not, define partitions and bounds before testing.

4.2 Writer attacks

For every authoritative state writer, determine:

* what state it can mutate;
* what preconditions it assumes;
* whether it validates those preconditions;
* whether it can bypass the transition boundary;
* whether it can leave partially committed state;
* whether it can perform persistence before validation;
* whether a thrown error leaves timers or persistent state inconsistent.

4.3 Boundary-bypass attacks

For every route identified in Phase 2:

* invoke it directly where possible;
* inject failures at its dependencies;
* inspect state before and after;
* verify invariant checks;
* verify safe failure;
* verify persistence ordering;
* verify timer state;
* verify reporting.

A route is not considered protected merely because a nearby route is protected.

4.4 Outer-layer failures

Test:

* draw callback exceptions;
* gamepad-reading exceptions;

* animation-frame scheduling behavior;
* `ResizeObserver` redraw failures;
* visibility-triggered pause;
* touch-drag transitions;
* storage-event handling.

SAF-5 requires drawing and gamepad errors to be reported without permanently stopping the relevant loop.

4.5 Protection attacks

Remove or weaken one protection at a time in scratch-only experiments.

Examples:

* remove `Number.isSafeInteger`;
* loosen score validation;
* remove current-piece fit validation;
* weaken queue validation;
* bypass post-action invariant checking;
* move persistence before or after validation;
* remove fault containment;
* weaken timer-state validation.

A surviving mutation is a candidate evidence gap. It must be evaluated for semantic relevance to the claimed property before being recorded as an evidence gap; a mutation that changes behavior only outside the requirement's semantic domain is not one. A mutant that cannot change behavior is handled as QA-6 requires: marked in the code with the reason.

It does not automatically establish a production defect.

No scratch mutation becomes a repository change without explicit classification and approval.

4.6 Circular-oracle attacks

For each important test:

* identify where the expected value comes from;
* determine whether that expectation is independently derived;
* verify that the test does not merely repeat production logic;
* verify that fixtures cannot silently reproduce the same faulty assumption;
* compare model and implementation independently where appropriate.

A test that calls the same validator as the implementation does not independently prove that validator.

4.7 Known-candidate reproduction order

The following reproduction order is mandatory unless a user-approved change records a different order.

1. F20: persistence ordering

Test each relevant persistence side effect:

* pause save;
* line-clear save;
* game-over forget;
* high-score write;
* New Game forgetting;
* any other authoritative persistence write.

For each:

1. establish a known-good persistent representation;
2. establish the corresponding valid in-memory state;
3. trace the actual ordering of state mutation, persistence side effects, invariant checking, and failure handling;
4. inject a failure at each reachable boundary between those operations;
5. record storage contents;
6. record engine state;
7. record fault reporting;
8. record timers;
9. determine whether the previous persisted representation survived;
10. compare behavior against O4 and the applicable requirement, and record whether any failure left the persisted representation different from the required last-known-good representation.

New Game must separately record whether the forgetting effect is intended to be a deletion, an invalidation or another effect, and whether any replacement persistent state exists.

---

2. F23: serialization failure

Inject a failure at serialization.

Record:

* whether serialization throws;
* whether `saveGameSession` catches it;
* what the caller observes (success or failure);
* what `runChecked()` observes;
* whether the invariant boundary runs;
* whether SAF-3 fault handling runs;
* whether game-over is reached;
* timer state;
* fault reporting;
* storage contents;
* preservation of the previous save.

Classify only after comparing with SAF-2, SAF-3, and O4. The existence of a catch is not itself a defect.

---

3. N1: New Game failure after the forgetting effect

Establish a valid saved state.

Inject a failure after the point where the forgetting effect is applied but before New Game has completed.

Record:

* whether the saved representation remains;
* whether a replacement representation exists;
* game state;
* timers;
* fault reporting;
* user-visible recovery path.

Classification depends on R2 and O4.

---

4. F24: storage-read failure and save discoverability

Inject a storage-read exception during Continue/save discovery.

Record:

* stored bytes/value before and after;
* cached saved-game availability;
* whether Continue remains available;
* whether a later successful read rediscovers the save;
* whether any write occurs;
* whether the game continues safely.

Classification depends on R4.

---

5. F21: sparse board predicate

Construct scratch-only cases including:

* `Array(20)`;
* a board with a deleted row;
* `Array(10)` as a row;
* sparse rows containing holes;
* otherwise valid dense boards.

Exercise the authoritative invariant boundary.

Record:

* whether each case is accepted;
* the exact invariant result;
* whether the representation is considered valid under R1;
* whether production writers can create the representation;
* whether any legal gameplay trace creates it.

These are three separate questions: does the predicate accept the representation, is the representation invalid under R1, and can a production transition introduce it. A reproduced acceptance is not a confirmed defect until the other two are answered.

A shallow predicate is not classified as a defect until its required validity semantics and reachability are established.

---

6. F22: gravity interval arithmetic

For all relevant levels, or an explicitly predeclared representative partition that covers the floating-point behavior:

1. calculate the exact mathematical interval required by PLY-2;
2. record the exact delay the implementation passes to the scheduler, and its exact numeric difference from that interval;
3. record how the browser converts that delay: `setTimeout` takes its delay as a WebIDL `long`, and that conversion truncates toward zero, so the converted value follows from the specification rather than from observation;
4. record observed firing and elapsed-time behavior as corroboration only, with the measurement method and tolerance declared in advance, since timers fire at or after their delay;
5. compare each of these with R3's accepted tolerance;
6. record whether any discrepancy is mathematical, in the requested delay, in the converted delay, or in observed timing.

The requested delay and the actual elapsed behavior are different claims. A floating-point representation alone does not establish a timing violation.

Do not classify before R3 is resolved.

---

7. Q1: malformed queue

Inject at the invariant boundary:

* `[null, null, null]`;
* a sparse length-3 array;
* `{ length: 3 }`;
* `[1, 2, 3]`;
* other representative non-piece values;
* any malformed canonical-piece representations identified by R1.

Record acceptance/rejection and whether production queue writers can create each case.

Classification depends on the exact R1 queue predicate.

---

8. Q2: malformed current piece and vacuous fit

Inject a current-piece representation whose cells collection is empty or otherwise malformed but causes a shallow fits predicate to return true.

Exercise the invariant boundary.

Record:

* whether the piece is accepted;
* whether it can participate in a transition;
* whether production constructors/writers can create it;
* whether R1 defines it as invalid.

Do not equate a vacuous predicate with a confirmed defect without the valid-state definition.

---

9. Q3: undefined timer handles

Exercise representative combinations of:

* playing + `gameLoopTask === undefined`;
* playing + `gameLoopTask === null`;
* resting + `lockDelayTask === undefined`;
* resting + `lockDelayTask === null`;
* both timers undefined;
* both timers present;
* game-over/paused states with timer variations.

Record invariant results and compare with SAF-4 and R1.

---

10. L1: line-clear evidence

First execute deterministic traces that legally produce:

* one line clear;
* multiple line clears where the engine supports them;
* save after clearing.

Verify:

* board transition;
* score;
* level;

* queue/bag behavior;
* persistence;
* invariant state.

Then re-run the previous bounded random trace exactly, preserving its seed and bound if available.

The previous observation was 3,738 locks with zero line clears. That observation is evidence only, not proof that the random path cannot clear lines. The permanent random play test performs no line clears, so it provides no evidence for line clear transitions under SAF-4 and SAF-6. Deterministic line clear traces provide that evidence; the random trace is supplementary and establishes nothing about correctness or impossibility.

If the random search is extended, record the seed and lock bound before execution.

---

11. P7: hand-edited `canHoldTetromino`

Construct a valid serialized game except for `canHoldTetromino = true` in a context that would otherwise allow another hold.

Record:

* parser result;
* reconstructed state;
* whether another hold is accepted;
* whether the behavior is considered within PLY-7 under R5.

This is not automatically a gameplay violation merely because it is possible through hand-edited storage.

---

12. Score invariant mutation

Use a scratch-only mutation replacing the safe-integer predicate with the weaker integer predicate.

Use an unsafe integer that is also a multiple of 100, such as 1e17.

Run:

* the permanent engine suite;
* coverage;
* mutation analysis as applicable.

Record whether the weakened predicate survives.

If it survives, evaluate it for semantic relevance to the score property, which depends on R1's score domain. If it is relevant, record an evidence gap, unless the actual requirements establish a production defect.

Do not weaken the shipped predicate; it is loosened only as REQUIREMENTS.md rule 2 allows.

---

13. S5: drawing failure

Inject an exception from the drawing path after an engine action has completed.

Record:

* whether the error is caught;
* whether the fault is reported as required;
* whether render caches are cleared;
* whether another frame is requested;
* whether later drawing attempts continue;
* whether engine state remains safe.

This is evaluated under SAF-5 and page-layer fault containment.

---

14. Scheduler fault cases

Phase 1 must explicitly record in the trust ledger whether scheduler registration and cancellation are inside or outside the trusted boundary before scheduler-fault results are certified.

Where scheduler behavior is inside the trusted boundary, inject:

Scheduling failure after callback registration

A scratch scheduler registers a callback/timer and then throws.

Record:

* returned handle;
* stored timer state;
* whether the callback can still execute;
* whether the engine reaches safe failure;
* whether an orphan timer remains capable of mutating state.

Cancellation failure

Make `clearTimeout` throw during failure cleanup.

Record:

* remaining timers;
* game state;
* fault state;
* whether callbacks can still mutate authoritative state.

If the scheduler is explicitly outside the trust boundary, these become trust-ledger assumptions rather than production defects. The assumption must be recorded and justified.

—

15. Entry-wrapper cases

Inspect the exact wrappers for:

* `handleAction`;
* `softDrop`;
* `cancelNewGame`;
* other public entry methods that merely invoke the guarded boundary.

For each wrapper:

1. identify every statement outside the guarded operation;
2. identify independent state mutation;
3. identify dependencies capable of throwing;
4. determine whether an exception can occur before or after the guard while leaving authoritative state changed;
5. test any real dependency boundary that can be injected.

If a wrapper contains no independent mutation or post-guard behavior and no relevant exception boundary exists, record the result as evidence rather than inventing a hypothetical failure.

—

4.8 Service-worker attacks (APP-2, APP-6)

public/sw.js keeps its own state: one cache per downloaded version (names starting `tetris-version-`), page pins in `tetris-pins`, and deletion of older versions. Test the following as attacks on APP-2 and APP-6, without presuming a defect:

* a cache from another project on the same origin that uses a name the worker deletes (`tetris`, `tetris-2`, or a name starting `tetris-version-`);
* a cache from another project on the same origin whose name starts with `tetris-version-` and is not empty: the worker counts any non-empty cache with that prefix as a complete version of the game, so record whether it is used to answer the game's requests, and which requirement that touches (APP-6's "touches only its own", and APP-2's rule that a page loads every file from the version it opened with);
* a cache name that starts with `tetris-version-` but has no number after it: record how the worker orders, selects, names and deletes caches when the number cannot be read (the version number is read with parseInt in public/sw.js), including the name given to the next download;
* a malformed or unreadable entry in `tetris-pins`;
* two downloads that start at the same time;
* a download cut off partway;
* a page opening just as a download finishes.

At c311c00 the browser tests in e2e/app.spec.js already cover offline use, clearing of the caches left by earlier versions, a foreign cache named `other-project` left alone (APP-6), a download that cannot complete, and a page that loads every file from the version it opened with while a newer one finishes (APP-2). Phase 4 records which attacks those tests already exercise and runs the ones that are missing as scratch tests, without adding permanent tests (a permanent test is a repository change that needs its own approval). The question is what this project's worker does when another project's or an unrecognised cache exists, and whether that satisfies APP-2 and APP-6. It is not a claim that GitHub Pages isolates origins, and it adds no cache-isolation requirement. Serving from a foreign cache is not by itself a violation of APP-6's "deletes only caches it made"; the audit records which requirement, if any, each observed behavior touches. If the deletion of the names `tetris` and `tetris-2` is reproduced and appears to conflict with APP-6, it is recorded as a Candidate finding and receives the Phase 4 disposition that every candidate needs; no defect is presumed. The same applies to any other behavior of the two new attacks that appears to conflict with an APP requirement; behavior that does not is recorded as evidence.

—

Falsification stopping rule

Every falsification target must have a predeclared domain.

For finite state/input classes, exhaustively enumerate the declared domain where practical.

For infinite or impractical spaces:

* define state partitions;
* define input partitions;
* define trace bounds;
* define random seeds;
* define lock/action/timer bounds;
* define the dependencies injected;
* define the failure points exercised.

Stop only when:

1. every declared case has been executed; or
2. a counterexample has been found; or
3. an environmental limitation prevents completion, and that limitation is explicitly documented, justified, and approved.

At completion, record:

* what was attempted;
* exact partitions and bounds;
* what was not attempted;
* why it was not attempted;
* whether the remaining space is covered by proof;
* whether it is covered by an explicit trust assumption;
* whether it remains an evidence gap.

An exhausted search is not proof unless the searched domain is itself the complete domain of the claim and the oracle/decision procedure is independently justified.

“No counterexample found” must never be converted into “proven” without that additional argument.

—

5. Classification and certification

5.1 Finding status

Finding status is independent of property certification.

Allowed finding states:

* Candidate
* Reproduced
* Not Reproduced
* Rejected
* Requirement Clarification
* Confirmed Defect
* Fixed
* Reverified

Not Reproduced means only that the specified reproduction did not reproduce the behavior.

Rejected means the concern was evaluated and does not violate the applicable requirement or adopted property.

A finding may require a requirement clarification while a related property remains Not Established.

A confirmed defect may coexist with an unresolved requirement question if the defect does not depend on that question.

—

5.2 Property certification status

Allowed property statuses:

* Not Assessed
* Not Established
* Tested Within Stated Bounds
* Proven by Reviewed Argument
* Proven by Exhaustive Enumeration
* Formally Verified
* Partially Established
* Out of Scope

Every Proven by Reviewed Argument claim must state its argument and assumptions.

Every Proven by Exhaustive Enumeration claim must state the complete domain, show that it was enumerated in full, and justify its oracle.

Formally Verified may be used only for an actual machine-checked formal proof.

—

5.3 Evidence record

Every final claim must identify:

* requirement/property;
* claim;
* method;
* assumptions;
* covered production code;
* covered model code if applicable;
* evidence;
* limitations;

* certification status;
* approver where approval is required.

---

6. Correction

No correction is implemented until:

1. the defect is classified;
2. the violated requirement/property is identified;
3. the smallest correction is identified;
4. the regression test is defined;
5. collateral behavior is considered;
6. the user/project owner approves the correction.

The correction must preserve all requirements not deliberately changed.

Each change is classified under REQUIREMENTS.md, “Classifying changes”: Refactor, Bug Fix, Intentional Change and/or Clarification, with the evidence that section requires for each class. The class is decided only after the behavioral effect is established.

One conceptual defect per PR, in the smallest commits and smallest PRs.

No architecture rewrite is justified merely because an alternative design appears cleaner.

6.1 Construction hypotheses

Potential corrections are evaluated against the following principles:

Area	Construction hypothesis
Transition boundary	One authoritative commit boundary should make legal transitions explicit and make bypasses detectable.
Validation	Validate at the authoritative boundary rather than relying on every caller to remember validation.
Persistence	Do not cross the O4 last-known-good boundary before the transition is sufficiently validated.
Fault handling	Engine failures enter one safe terminal behavior without leaving active timers.
Model	Keep the reference model independent from production transition/validation logic.
Input	Preserve one-shot/held distinctions and explicit modifier handling.
Rendering	Rendering failures remain outside the engine's authoritative state transition.
Tests	Preserve regression, coverage, mutation, and independent-oracle value.

These are hypotheses, not automatic implementation decisions.

---

7. Implementation

Implementation begins only after the Audit Advancement Gate is satisfied and the user/project owner has explicitly approved the relevant production change.

For each approved change:

1. make the smallest production modification;
2. add or preserve the required regression test;
3. run focused tests;
4. run the relevant complete suite;
5. run coverage;
6. run mutation analysis as required;
7. run browser tests;
8. run type checks;
9. inspect the diff for unrelated changes.

Existing checks must remain intact; a runtime check is loosened only as REQUIREMENTS.md rule 2 allows.

Every PR states its REQUIREMENTS.md change class with the evidence that class requires, ticks the class in the PR template, and records in AUDIT.md, in the same PR, any finding it confirms, fixes or rejects and any verification decision it makes.

No unrelated cleanup is bundled into a correctness fix.

---

8. Re-prove and re-falsify

Every production correction requires both:

* proof/evidence that the intended property now holds;
* re-falsification of the original counterexample.

The original regression must fail against the pre-fix implementation and pass against the corrected implementation.

The correction must also be tested against failure modes plausibly affected by the correction, so the fix does not merely move the defect. This is bounded to the corrected property and the protections the correction could have affected; it does not reopen the full audit.

For changes affecting the trusted boundary, repeat the relevant boundary-bypass analysis.

For changes affecting persistence, repeat O4 ordering tests.

For changes affecting invariants, repeat the malformed-state attacks.

For changes affecting timers, repeat timer-state and timing evidence.

For changes affecting the model, re-establish O8 independence and correspondence.

—

9. Final certification

Certification is requirement-by-requirement.

For each applicable requirement:

1. identify the claim;
2. identify the production code establishing it;
3. identify the proof/test/model evidence;
4. state assumptions;
5. state limitations;
6. assign certification status;
7. identify remaining evidence gaps;
8. identify the approving decision where required.

A requirement is not certified merely because the test suite is green.

A green suite is evidence at its tested scope.

A mutation score is evidence about test sensitivity to the mutations exercised, not a universal correctness proof.

A reviewed argument is not a machine-checked proof.

—

10. Minimize trusted core and permanent tests

10.1 Trusted-core minimization

After correctness is established, minimize the amount of code that must be trusted.

The target is not minimum line count.

The target is minimum code whose correctness must be assumed rather than independently demonstrated.

For every trusted component:

* identify responsibility;
* identify assumptions;
* identify evidence;
* identify whether the component can mutate authoritative state;
* identify whether it can bypass the transition boundary;
* remove unnecessary authority where practical.

—

10.2 Permanent-test minimization

Existing protections and tests remain unless the audit establishes that their required protection value is preserved.

Unchanged requirements

When requirements are unchanged, a test may be removed only if another permanent test establishes the same outcome for the same required property, for the same relevant input/state, at the same authoritative boundary, while preserving:

* required line/branch/function coverage;
* mutation effectiveness;
* regression value;
* independent-oracle value;
* the specific safety property previously protected.

The fact that another test happens to execute the same code is insufficient.

Changed requirements

If a requirement changes, REQUIREMENTS.md rule 2 governs whether the old behavioral requirement/test should be changed, replaced, or removed.

The audit minimization rule does not override a deliberate requirement change.

Even after a requirement change, required safety protections, coverage obligations, mutation effectiveness, and necessary regression evidence must not disappear accidentally.

Every removal must state:

* what protection the old test provided;
* what replaces that protection;
* why the replacement is sufficient;
* what requirement authorizes the change;
* whether coverage/mutation gates remain satisfied;
* user/project-owner approval where required.

Current audit finding record

The following records are carried forward from prior audit work.

F17

Status: Rejected

A stored high score such as 150 is accepted and shown, while a saved game with a score of 150 is rejected.

Disposition: Rejected, Harmless.

The game only stores real scores, so such a high-score value requires manual storage editing.

High score is displayed and compared but does not enter game state. It is replaced when a real score exceeds it.

Values that are not whole numbers ≥ 0 are treated as zero.

No requirement says high scores must be multiples of 100.

No change.

F18

Status: Rejected

Concern that invariant checking after game over/fault could repeatedly report or otherwise mishandle an already-faulted state.

SAF-4 requires an invariant check after every action and timer, including after game over.

Continuing the check preserves detection of regressions that mutate a stopped game.

Skipping the check would create a blind spot.

AUDIT.md F18 records that after such a fault later actions report the same fault again because invariants are still checked at game over. This observation is not itself relied upon as proof.

No change.

F19

Finding status: Fixed, historically (AUDIT.md F19, PR #42). Reverification under this audit is pending; no current verification is claimed here.

The invariant checker was originally outside the `runChecked()` catch boundary. If the checker itself threw, the error escaped, the game could continue, no fault was reported, and the screen was not redrawn.

PR #42 moved the operation and invariant checker into the existing catch boundary.

This was a concrete production defect.

PR #42 and AUDIT.md record the correction as made and merged. That is historical repository evidence. Deployment, and the current behavior, are not verified by this blueprint; current verification is pending Phase 0 and Phase 8 evidence.

Reverification must retain the regression test proving that an invariant-check failure enters safe failure.

F20

Status: Candidate pending independent reproduction

Candidate concern: persistence side effects, where preservation is required, may occur before the transition has reached the required post-transition validation boundary. The implementation ordering is an observation to reproduce; whether it is a defect is not yet concluded.

F20 is a candidate root mechanism, not a presumption that all listed persistence operations constitute one defect. If different operations violate different requirements or require materially different corrections, they receive separate finding IDs at classification.

The following persistence effects are reproduction classes under F20. They are not presumed affected operations and not presumed to share one defect:

* pause save;
* line-clear save;
* game-over forgetting;
* high-score write;
* New Game forgetting;
* other persistence effects.

This is evaluated primarily under SAF-3 and O4.

F20 asks four separate questions.

Ordering: did persistence happen before the required post-transition validation boundary?

Applicability: does the relevant requirement require preservation of the persisted representation for this effect? O4 applies only where preservation is required, so not every persistence write is tested as though it does.

Correspondence: does the persisted save hold the committed state the requirement calls for (for example the paused game, or the game after a line clear), and not a state from the middle of a step? This is examined separately from ordering and from preservation.

Consequence: did a failure violate that preservation requirement, by causing the required last-good representation to be lost or replaced?

An early write is not by itself a violation if the requirements permit it and the resulting persisted representation remains good under O4. Whether a save written during a step that then faults counts as the last good save is decided under R4, so a valid-content save written by a step that later faults is not assumed to be the last good save. The defect question is whether a failing transition can cross the last-known-good persistence boundary before the failure is detected, with the consequence established independently of the ordering.

The high-score write is traced because it is an authoritative persistence side effect. It is not presumed to be subject to the saved-game last-known-good rule; that is decided under R4.

R2 determines the forgetting semantics for New Game (STA-1) and game over (STA-3).

—

F21

Status: Candidate pending R1 and independent reproduction

The board invariant uses `Array.prototype.every()`, allowing sparse arrays to satisfy predicates that inspect only existing elements.

Potential cases include:

* a 20-element sparse board;
* a deleted row;
* a sparse row;
* other holes.

Production writers currently do not appear to create such holes.

The classification depends on the exact R1 valid-state definition and whether the representation can reach the authoritative state boundary.

—

F22

Status: Candidate pending R3 and independent reproduction

Gravity interval arithmetic can produce floating-point values slightly below nominal mathematical milliseconds, such as approximately 499.99999999999994 ms.

Whether the browser fires such a value early is not established by the arithmetic alone. The browser's conversion of the delay follows from the WebIDL specification; observed firing times only corroborate it (4.7).

The implementation fact must be separated from the requirement classification.

R3 determines the accepted timing tolerance.

—

F23

Status: Candidate pending independent reproduction

`saveGameSession` catches serialization failures in a way that may allow an engine failure during persistence to be swallowed rather than entering SAF-3.

This is evaluated together with O4 but remains a separate finding until reproduction establishes the exact failure boundary and behavior. If F23 is shown to be solely a mechanism by which the F20 persistence-ordering violation occurs, it may be classified as a manifestation of F20 rather than a separate defect. If it independently violates SAF-2 or SAF-3 without an O4 ordering violation, it remains separate.

—

F24

Status: Candidate pending R4 and independent reproduction

A storage-read failure during Continue can clear the saved-game availability flag even though the underlying stored value remains.

R4 determines whether that loss of discoverability violates the storage-failure requirement.

Q1

Status: Candidate pending R1

The queue predicate may check only `length === 3`, allowing malformed values with length three.

Examples:

* three nulls;
* sparse length-three arrays;
* `{ length: 3 }`;
* `[1, 2, 3]`;
* other non-piece values.

The production queue writer creates canonical pieces.

R1 determines whether the invariant must reject these representations.

Q2

Status: Candidate pending R1

A malformed current-piece object with no cells may satisfy a vacuous fit predicate.

R1 determines whether current-piece validity includes canonical piece structure independently of geometric fit.

Q3

Status: Candidate pending R1

Timer predicates using `!== null` may treat undefined as an existing timer.

Potential violations include:

* playing without gravity;
* resting without lock delay.

R1 and SAF-4 determine the exact required timer-state predicate.

L1

Status: Candidate. Evidence Gap: open.

A prior random-play probe produced 3,738 locks with zero line clears.

The permanent random play test therefore provides no evidence for line clear transitions under SAF-4 and SAF-6; that evidence comes from targeted tests and deterministic line clear traces. The random trace is supplementary and establishes nothing about correctness or impossibility.

The audit requires a deterministic trace plus the recorded bounded random trace before closing the evidence question.

N1

Status: Candidate, potentially folded into F20

A fault during New Game could occur after the saved-game flag is set to false, so that the previous save is no longer offered. At the repository commit checked (30e0085), the stored save text is not deleted by this effect (`isSessionSaved = false` at `gameManager.js:194`; the only `removeItem` is in the in-memory fallback storage). The previous stored representation therefore remains but is no longer offered, as in F24.

R2 determines whether this violates the intended New Game semantics.

N1 stays a separate record cross-referenced to F20. R2 can make N1 a non-defect even if F20 is confirmed, so the two are merged only if reproduction and R2 show the same root cause and the same outcome; the final record may then cross-reference rather than duplicate the correction.

P7

Status: Candidate pending R5

A hand-edited valid save with `canHoldTetromino = true` may allow another hold after reopening/Continue.

This is not automatically a PLY-7 violation. Whether hand-edited persisted state counts as legal gameplay is R5's decision (1.4).

R5 determines the applicable scope.

—

Score

Status: Candidate. Evidence Gap: possible.

Replacing `Number.isSafeInteger(score)` with `Number.isInteger(score)` in a scratch mutation permits unsafe integer scores.

An unsafe integer that is a multiple of 100, such as 1e17, is the relevant counterexample class.

If the permanent suite does not detect the weakened predicate, the result is primarily a test-strength evidence gap unless a requirement violation is separately established.

The shipped safe-integer check is loosened only as REQUIREMENTS.md rule 2 allows. This audit does not make `Number.isSafeInteger` normative; whether unsafe integers are valid scores is R1's question.

—

S5

Status: Candidate pending reproduction

A page redraw callback can throw after a checked engine step.

The audit must establish whether `drawSafely` contains the required containment and loop continuation behavior under SAF-5.

—

Scheduler fault cases

Status: Candidate; classification depends on the O6 trust ledger.

Potential failures include:

* scheduler registration followed by a throw;
* cancellation throwing during `failSafe`.

These are classified according to the final O6 trust ledger.

If the scheduler is explicitly outside the trusted project boundary, the result is an assumption to record rather than an invented production defect.

—

Entry-wrapper cases

Status: Candidate pending boundary inspection

Public wrappers `handleAction`, `softDrop` and `cancelNewGame` must be inspected for state mutation or failure boundaries outside the authoritative guard.

Observed at 30e0085: `softDrop` sets `isConfirmingNewGame = false` before calling the guard (gameManager.js line 528); `handleAction` and `cancelNewGame` contain only the guard call (lines 459 to 461 and 533 to 537). The previous audit's statement that all three contain only the guard call was wrong for `softDrop`.

If a wrapper contains only the guarded call and no independent mutation or post-guard operation, no hypothetical fault is invented.

—

Trust ledger

ID Property/component Trust claim Evidence required
O1 Transition boundary All authoritative state commits pass through the defined boundary or an explicitly authorized exception. Boundary inventory, route analysis, bypass tests.
O2 Valid transitions Legal transitions preserve valid state or enter defined safe terminal state. Reference model, invariant proof/evidence, differential tests, legal-play tests.
O3 Persistence validation Untrusted persisted input cannot partially become authoritative state. Session parser analysis, malformed fixtures, parser tests, differential evidence.
O4 Persistence ordering Required last-known-good representation is preserved across relevant failures. Failure injection, storage inspection, R2 and R4 decisions.
O5 Code-to-claim link Every certification claim maps to shipped implementation code and independent evidence. Traceability records.
O6 Trusted components Assumptions about schedulers, browser APIs, storage, rendering, and other external components are explicit. Trust ledger and failure analysis.
O7 Boundary enforcement Production routes cannot bypass the authoritative transition boundary without an approved exception. Route inventory and deliberate bypass tests.
O8 Independent model/correspondence The executable model is independently constructed and its correspondence to production is independently evidenced. Model source, independence argument, differential tests.

—

Apple/security research grounding

External security architecture and engineering guidance, when a source is cited in Appendix A, may be used to justify adoption of a security property. Appendix A cites sources only for the themes they support. A theme with no cited source, and every property no cited source supports, rests on the owner's adoption alone.

Research grounding can justify adoption of a security property, but cannot establish that this implementation satisfies that property.

Implementation satisfaction must be established by the project's own requirement analysis, proof method, testing, model correspondence, trust assumptions, and evidence.

Platform-specific Apple mechanisms are not copied into the project merely because they appear in Apple documentation.

Only the underlying applicable security property is adopted when the project lacks the relevant platform mechanism.

___

Appendix A. Research grounding

This appendix cites the external sources listed below, each for the theme and claim stated and no wider. No Apple source is cited. Assistant-written secondary summaries are kept as files listed in Appendix D; they are kept for reference and are not cited as grounding. The sources are design-grounding material for these themes, which are the topics the adopted properties address:

* trusted boundaries;
* validated state transitions;
* least authority;
* fail-safe behavior;
* persistence ordering;
* separation of trusted and untrusted data;
* independent validation;
* defense against malformed state.

They are not implementation certification.

Cited sources (read October 4, 2026; each states what it supports and where it stops):

* Separation of trusted and untrusted data; defense against malformed state; validated state transitions (O2, O3). OWASP Input Validation Cheat Sheet, https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html: validate syntax and semantics, accept by allowlist, and parse then validate before processing or storage. It is written for server-facing input. It supports treating a saved game as untrusted input; it does not define this game's valid state. OWASP HTML5 Security Cheat Sheet, https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html: data in Web Storage can be altered by anyone with local access, so authentication or authorization must not be assumed over it. It does not prescribe a validation procedure.
* Least authority. GitHub Docs, Secure use reference, https://docs.github.com/en/actions/reference/security/secure-use: least privilege for tokens and secrets, and pinning an action to a full-length commit SHA as the most secure option, or to a tag only if the creator is trusted. It applies to the CI workflows, not to the game code.
* Persistence ordering; fail-safe behavior (O4). WHATWG HTML Standard, Web Storage, https://html.spec.whatwg.org/multipage/webstorage.html, and WHATWG Storage Standard, https://storage.spec.whatwg.org/: storage is scoped to an origin; the browser may clear storage in its default mode; persistent mode needs a permission grant and, when granted, protects against the browser's clearing policies, though the user can still clear it. Whether a browser grants the permission varies. These sources describe what storage can lose. They do not give a write-ordering method. WebKit Tracking Prevention, https://webkit.org/tracking-prevention/: Safari's ITP deletes script-writable storage after 7 days of no user interaction with the website, and the page exempts the first-party domain of home screen web applications from that cap. It is Safari-specific. How Safari treats persistent storage was not verified.
* Boundary enforcement; trusted boundaries (O1, O7). W3C Service Workers, https://www.w3.org/TR/service-workers/, and the Chromium Service Worker Security FAQ (listed in C6): service worker scope, same-origin rules and cache behavior that bear on the cache attacks in Phase 4. They do not define this project's authoritative transition boundary.
* General secure-development practice. NIST SP 800-218, https://csrc.nist.gov/pubs/sp/800/218/final, OWASP ASVS 5.0, https://asvs.dev/v5.0.0/Preface/, and OWASP Top 10 A04 Insecure Design, https://owasp.org/Top10/A04_2021-Insecure_Design/: general frameworks for secure design, development and verification of web applications. No numbered practice or requirement of them is mapped to a theme here. Many ASVS requirements assume a backend that this project does not have.
* No source was verified for independent validation (O5, O6, O8) or for the write-ordering method in O4; those properties rest on the owner's adoption alone.

Platform limits that bear on the trust ledger (Appendix E), cited as limits and not as themes: MDN, Content Security Policy, https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP, and its frame-ancestors page, https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Security-Policy/frame-ancestors (a policy set by meta tag does not support frame-ancestors or report-only); MDN Subresource Integrity, https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity (it detects a changed fetched resource; it does not protect against a local player or a malicious change by the repository owner; a checksum detects accidental corruption and is not tamper protection).

The project does not claim conformance to a platform security architecture merely because analogous properties are adopted.

___

Appendix B. Baseline evidence

B1. Values to record at Phase 0 closure

Record:

* audited commit;
* `GameManager` lines;

* mutable-state inventory;
* test count;
* browser-test count;
* coverage (engine and page scripts, each as lines, branches and functions);
* mutation totals;
* killed/survived/timed-out/errored mutant counts;
* mutation score and repository threshold;
* type-check result;
* CI result;
* deployment result where applicable;
* relevant dependency and repository configuration facts;
* service-worker/network observations;
* hosting/security-header observations relevant to APP-6.

The approved Phase 0 execution record is `docs/phase-0-baseline.md`. D13 in AUDIT.md approves Phase 0 against that checksum and pull request 46, and CI run 37222249141 (run number 104). Phase 0 values are not copied into this blueprint; the record is the single place that holds them.

B2. Existing observations from 30e0085

Observed so far:

* `GameManager` (public/game/gameManager.js) is 591 lines.
* The constructor assigns 24 fields (lines 84 to 115); 9 of them are dependencies or settings assigned only there (`storage`, `scheduler`, `factory`, `onFault`, `onChange`, `rows`, `columns`, `maxLockDelayResets`, `lockDelayInterval`).
* The mutable-state representation permits combinations that are not necessarily semantically valid.
* `findInvariantViolation()` is concentrated around lines 411-428.
* `parseSession` is the central persisted-state parser.
* `npm run typecheck` exists.
* The service worker's observed network calls are the passthrough `fetch(request)` around public/sw.js line 100 and the download of every app file by `cache.addAll` (line 142), which fetches each file from the network.
* No server/shared leaderboard is present in the audited application architecture.
* Main deployment follows CI.
* Repository rules require the test check but do not necessarily require a PR for every change.
* The runtime dependency `express` is local-development infrastructure.
* One constant `innerHTML` use exists.
* No project-level CSP was established from the existing audit.
* Hosting-header limits have not yet been established.
* GitHub Pages shared-origin/storage/cache implications remain relevant to APP-6.

B3. Mutation baseline

A prior local mutation run at 30e0085 recorded:

* approximately 67 minutes;
* 1,152 mutants;
* 1,120 killed;
* 32 timed out;
* 0 survived;
* 0 without coverage;
* 0 errors;
* reported mutation score: 100.00.

This is baseline evidence, not a correctness proof.

Timeouts are counted according to the repository mutation gate, but a timeout is weaker evidence than an assertion that directly kills a mutant.

The required CI result must be reproduced and recorded during Phase 0 closure.

B4. Browser baseline

WebKit was not runnable on the review machine at the time of the prior audit.

Safari/WebKit coverage therefore requires the designated CI or equivalent environment.

Chromium browser coverage and iPhone-size testing remain required under QA-4.

B5. Prior evidence not independently re-run

The following prior-session evidence must not be treated as independently reproduced until rerun where required:

* fault-injection matrix;
* persistence write detector;
* random-play probe;
* browser fault probes;
* relevant timing measurements.

—

Completion standard

The audit is complete only when:

1. every applicable requirement and every adopted security property (including each adopted O-property) has an explicit final status and an evidence record appropriate to its nature (for example a property claim for O1 to O4 and O7, traceability for O5, assumption accounting for O6, and model independence and correspondence for O8);
2. all R1-R5 decisions are recorded and approved;
3. the authoritative transition boundary is independently defined and inventoried;
4. the trust ledger is complete;
5. all confirmed defects have approved minimal corrections;
6. every correction has a pre-fix failing regression;
7. corrected behavior is reverified;
8. original counterexamples have been re-falsified;
9. declared falsification domains and bounds are exhausted where falsification is the applicable evidence method, or the remaining domain is covered by a claim certified Proven under 5.2, or each limitation is documented, justified and approved by the user/project owner. Executable-model or differential agreement alone does not close an untested domain;
10. every remaining limitation is recorded;
11. every certification claim names its method, assumptions, covered code/model, evidence, and limitations;
12. QA requirements are satisfied at their required scope;
13. permanent tests are minimized only under the test-removal policy;
14. no unsupported claim of formal verification is made;
15. the user/project owner approves the final certification and any resulting repository changes.

Final principle: the audit does not certify that no bug exists in an unlimited universe. It certifies only the claims whose stated domains, assumptions, proof methods, tests, models, trust boundaries, and evidence have actually been established.

—

Appendix C. Published Tetris references and what they do not establish

Source: the research survey "Tetris rules and implementations: research for the tetris-js audit", version 2, October 4, 2026 (Tetris-rules-research-v2.pdf, kept at docs/sources/Tetris-rules-research-v2.pdf), which this appendix summarizes. The audit has not independently re-read these sources or re-run its comparisons. Where a statement below is a reading of a source, it says so.

C1. Status and limits

* This appendix is reference material. It is not a requirement, it adopts no rule, and it is not evidence that tetris-js satisfies any requirement or property. REQUIREMENTS.md at 30e0085 decides what counts as a defect.

* A difference between tetris-js and a published game is not a finding. It becomes a finding only if it breaks a REQUIREMENTS.md requirement. Adopting a published rule as a new requirement is an Intentional Change, made in REQUIREMENTS.md together with its code; it is not a finding.

* Coverage is a representative sample, not "all public versions of Tetris", which is not a finite list. Read in full or in the relevant parts: the 2009 Tetris Design Guideline (a leaked, unauthenticated copy read as archive.org OCR text, so it may contain OCR errors and may differ from later versions), community wiki summaries, the source of three public engines, one formally analyzed Tetris, and web-platform security guidance. README level only: tetr.js and tetrs. No engine was run, no other engine's tests or lock delay, hold and scoring code was read, and the Tetris Company's current rules are not public.

* Conformance to any public version does not show the game is bug-free or secure, and passing every test does not either.

C2. Published rules compared with REQUIREMENTS.md

Every statement about published games in this section is what the supplied survey reports; none is an audit fact. Requirement IDs in the tetris-js column were checked to exist in REQUIREMENTS.md at 30e0085, and two misattributions were corrected; that column's readings of the code are the survey's. Class: Matches, Variation (published games differ among themselves), Yours (a REQUIREMENTS.md rule differs), Unspecified (neither the requirements nor the tests describe it). The supplied research survey reports the published statements below from the sources in C6. The Matches, Variation and Yours classes below are observations reported by the supplied research survey. They are not independently verified by this audit and cannot support certification until independently reproduced where applicable.

Topic | Published (as reported by the survey) | tetris-js (requirement) | Class (survey-reported, not an audit finding)

Playfield | 10 x 20 matrix plus an invisible buffer; spawn on rows 21 and 22 (2009 Guideline); spawn row varies by game. | 10 x 20, nothing outside it (PCE-1). Spawn in the top two rows (PCE-3). | Variation, and Yours. The supplied research survey interprets this difference as meaning that kicks above the top are blocked and that this implementation therefore lacks that form of lock-out (an inference from the reported playfield and spawn rules, not a reported observation).

Spawn columns | Three-wide pieces on cells 4 to 6 (2009 Guideline 3.4). | Columns 3 to 5 zero-based, I at 3 to 6, O at 4 to 5. | Matches, by the survey's reading of the code.

Wall kicks | Five tests per transition; JLSTZ share one table, I has its own, O does not kick (SRS page). | Same tables (PCE-4). | Reported as matching (C3, inherited, unverified).

Failed rotation | "If all five rotations fail, the rotation cannot occur" (2009 Guideline, Appendix A). NullpoMino, term-tetris and python-tetris also leave the piece unchanged. | After all five kicks fail, tries the opposite direction (PCE-4). | Yours. Not in the Guideline text or the three engines read.

Randomizer | 7-bag: shuffle seven, deal, refill. | 7-bag, fresh bag per game (PCE-5). | Matches.

Lock delay | Extended Placement: 0.5 s, 15 moves or rotations, count resets when the piece falls one row below the lowest yet reached, immediate lock after 15 (2009 Guideline 5.7). A rotation that lifts the piece pauses the timer without resetting it. | 0.5 s, 15 resets, count resets on a lower row (PLY-6). The delay restarts on a lifting rotation. | Matches, except the lifting-rotation nuance, which differs. Not a defect under REQUIREMENTS.md rules 1 and 3.

Gravity and level | Marathon curve (0.8 - (level - 1) x 0.007)^(level - 1) s per row; level rises by line clears. | max(0.25, 0.7 - 0.02 x (level - 1)) s (PLY-2); level = floor(score / 1000) + 1 (SCO-2). | Yours.

Scoring | Guideline-era games: 100/300/500/800 x level plus T-spins, back-to-back, combos, drop points, perfect clears. | 100/300/500/800, no multiplier, nothing else (SCO-1). | Yours. Base values match. Any drop-point scoring would first need the implementation's multiple-of-100 score check to change. No requirement states that rule; whether SAF-4's "valid score" includes it is R1's question.

Hold | One held piece; swapped-in piece starts north facing; a lock down between holds. | Hold once per piece, returns in its starting state (PLY-7). | Matches.

Game over | Lock out, block out, or top out (2009 Guideline section 10). | Game over when a new or held piece has no room (STA-3). | Yours. Block out matches.

Controls and repeat | Hold on C or Shift; DAS 167 ms and ARR 33 ms recommended. | Hold on H; 167 ms then 33 ms (INP-1, INP-2). | Yours (hold key); Matches (repeat).

Soft drop | 20 times the normal fall speed (2009 Guideline 5.5). | One row per press (PLY-3); a row every 50 ms while held (INP-2). | Variation.

Entry delay, line-clear delay, T-spin, garbage, perfect clear | Present in many Guideline games. | Not in the requirements. | Unspecified. Absence is not a defect (rule 3).

Historical context: NES Tetris has no wall kicks or lock delay and can crash after about 1,550 lines from inefficient multiplication (NES Tetris, Wikipedia). The survey's reasoning that tetris-js cannot reach a comparable overflow (score checked with `Number.isSafeInteger` and as a multiple of 100) is a claim to confirm in Phase 1, not a finding.

C3. Kick-table comparison

The supplied research survey reports the results in this section; they are inherited and are not independently verified by this audit; the audit has not re-run the comparison. Method as reported: each engine's kick table was converted to the wiki's (x, y) convention and compared by script, all eight transitions per table, with the SRS tables at tetris.wiki/SRS. tetris-js was compared the same way from tetrominoFactory.js and tetromino.js at 30e0085, with counterclockwise kicks derived as rotate() does. No engine was run.

Implementation | Read | Kick data vs wiki SRS (survey-reported, unverified) | Other points (survey-reported)

tetris-js | tetrominoFactory.js, tetromino.js | 16 of 16 match | Opposite-direction fallback after all kicks fail (PCE-4): its own rule.

NullpoMino (Java) | StandardWallkick.java | 16 of 16 match | Returns no result on failure, so the rotation does not happen. Also has 180-degree kick tables and an allowUpward flag.

term-tetris (Rust) | src/tetromino.rs | 16 of 16 match | Restores the old rotation on failure. A small project; agreement shows copying fidelity, not independent derivation.

python-tetris (Python) | tetris/impl/rotation.py | 15 of 16 match | One I-piece entry (2 to R, fifth test) reads (row -1, col +2) where the wiki converts to (row -1, col -2). Looks like a sign typo. A lead, not a confirmed bug: not run, tests not read, not reported upstream.

AdaCore SPARK Tetris (Ada) | Two blog posts only | Not Checked | Proves absence of run-time errors (array bounds, ranges) on about 165 lines of game logic. The article does not claim rule correctness.

tetr.js, tetrs | README only | Not Checked | Guideline and SRS compliance claimed, not verified.

* The supplied research survey reports that the tables in tetris-js agree with the published tables (inherited, unverified). That would check the numbers only.

* Agreement among implementations is weak evidence: they copy the same wiki or Guideline, so matches are not independent checks.

* A published-table comparison can catch a transcription slip that 100% coverage would not, because coverage shows that a line ran, not that a number is right. This is an instance of the circular-oracle concern in 3.2 and 4.6.

* Not shown: whether the rotation logic applies the data correctly, collision at walls and the floor, and how the PCE-4 fallback behaves in play.

C4. Security and robustness references

Source | What it says (as reported) | Bears on

OWASP HTML5 Security Cheat Sheet, Local Storage | localStorage is shared across the origin, can be loaded with malicious data by a single XSS, and should not be trusted. No path scoping exists. The sheet advises against hosting several applications on one origin. | SAF-1, SAF-2, APP-6. Keeps open the question of who else can write this origin's storage.

MDN, Window.localStorage | Values are strings; access can throw if blocked by policy; data persists with no expiration; origin-wide. | SAF-2.

Chromium Service Worker Security FAQ | A registration lasts indefinitely, updates are checked on navigation, and registration matches by serialized URL prefix, so sites should be careful about which paths can serve worker scripts. | APP-2, APP-3, APP-6: sw.js on a shared origin is a boundary worth stating in Phase 1.

V. Romeo, "implementing secure leaderboards" (not Tetris) | A client-sent score can be spoofed; deterministic play from a seed plus an input log lets a server replay it. | The game has no server or leaderboard, so score spoofing is out of scope. A seeded replay is a possible audit instrument because the random source and scheduler are injectable (PCE-5).

The supplied research survey reports that no published security standard specific to Tetris games was identified. The supplied research survey identifies the following classes as relevant to a client-only game: untrusted stored data, storage faults, a stale or malicious service-worker cache, same-origin neighbors on GitHub Pages, and input and timing faults. The SAF and APP requirements already name these.

C5. Items the research surfaces for Phase 1 (not decisions)

None of these is adopted. Each is the owner's decision under the Audit Advancement Gate.

* Whether the published SRS tables and the C3 comparison become the oracle for kick data, and whether the 16-transition check becomes evidence for PCE-2 and PCE-4. That is a new proof obligation only if adopted.

* Whether the intentional differences in C2 (the PCE-4 fallback, no hidden buffer, level from score, the gravity curve, scoring without a multiplier, and PLY-6's restart on a lifting rotation) are recorded in AUDIT.md as "not a finding".

* Which independent oracle to use for kicks, hold, lock down and game over, if any: a published-table check, a second implementation from the SRS spec, or differential testing against an open-source engine. Each establishes a different thing.

* Whether a seeded replay is used as an audit instrument.

C6. Sources

* Tetris Guideline: https://tetris.wiki/Tetris_Guideline; SRS and kick tables: https://tetris.wiki/SRS; Random Generator: https://tetris.wiki/Random_Generator; Lock delay: https://tetris.wiki/Lock_delay; Scoring: https://tetris.wiki/Scoring; Marathon: https://tetris.wiki/Marathon; Top out: https://tetris.wiki/Top_out

* 2009 Tetris Design Guideline (leaked, unauthenticated copy; OCR text): https://archive.org/details/2009-tetris-variant-concepts_202201

* NullpoMino StandardWallkick.java: https://github.com/nullpomino/nullpomino/blob/4de098dd0b48d991247313d8dba30b9721e6f9d9/nullpomino-core/src/main/java/mu/nu/nullpo/game/subsystem/wallkick/StandardWallkick.java

* term-tetris: https://git.pvv.ntnu.no/vegardbm/term-tetris/raw/branch/master/src/tetromino.rs; python-tetris: https://github.com/g3ner1c/python-tetris

* AdaCore: https://www.adacore.com/blog/tetris-in-spark-on-arm-cortex-m4 and https://www.adacore.com/blog/spark-tetris-on-the-raspberry-pico

* tetr.js: https://github.com/simonlc/tetr.js/; tetrs: https://github.com/tiehuis/tetrs/

* Guideline compliant game differences: https://www.tetris.wiki/Guideline_compliant_game_differences; Rotation systems: https://strategywiki.org/wiki/Tetris/Rotation_systems; NES Tetris: https://en.wikipedia.org/wiki/Tetris_(NES_video_game)

* OWASP HTML5 Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html; MDN localStorage: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage; Chromium Service Worker Security FAQ: https://chromium.googlesource.com/chromium/src/+/main/docs/security/service-worker-security-faq.md; Romeo: https://vittorioromeo.com/index/blog/oh_secure_leaderboards.html

Source types, read October 4, 2026. No link above was replaced. Community or tertiary sources: the tetris.wiki, www.tetris.wiki and strategywiki pages and the Wikipedia page. Unauthenticated copy: the archive.org 2009 Guideline. Open-source code: NullpoMino is pinned to a commit; term-tetris is read from a branch, and python-tetris, tetr.js and tetrs are unpinned pages whose content can change. At the time of reading, the python-tetris URL resolved to a page titled skysomorphic/python-tetris. Vendor technical blogs: the AdaCore pages. Authoritative for what they state: the OWASP, MDN and Chromium pages. Personal blog: Romeo. No source supporting the same claims as the Tetris rules sources or the Romeo blog was verified, and none is claimed to exist.

—

Appendix D. Source files kept in the repository

These files are kept for reference. The blueprint relies only on the Tetris survey, through Appendix C; the Apple and Google summaries are background and are not relied on. Where the Tetris survey and Appendix C differ, Appendix C governs. The text file docs/audit-blueprint.txt is authoritative; the PDF is a rendering of it. The other files are assistant-written secondary summaries prepared for the owner. They are not audit facts, not evidence that the implementation conforms to anything, and not the wording of Apple, Google or any other source; the primary sources they cite remain the authority for what those sources say.

File | What it is | Provenance | SHA-256

docs/audit-blueprint.pdf | PDF rendering of docs/audit-blueprint.txt | Rendered from the text file; the text file is authoritative | See the AUDIT.md ledger row (it cannot be listed here, because it renders this file)

docs/sources/Tetris-rules-research-v2.pdf | Assistant-written research survey, version 2, October 4, 2026 (summarised by Appendix C) | secondary summary; cites published Tetris rules and open-source engines | 47db35b0207d7867130adf908eb5a0ece6834e606d4ad21819aabf05c587d0ef

docs/sources/Apple-platform-security-review.pdf | Assistant-written summary of Apple's published security documentation, October 3, 2026 | secondary summary; primary sources are Apple's Platform Security guide and security.apple.com publications it cites | e561334723cff73ad4135cb0f16a1d9880ea9dad403ce4db52da6e2917c5373f

docs/sources/Apple-platform-security-full-reading.pdf | Assistant-written fuller reading of Apple's published security documentation, October 3, 2026 | secondary summary; primary sources are the Apple pages it cites | b543029911457180a27f71500a7353d6b980a9b10e17d6f5163b0372a3ac5ab3

docs/sources/Google-security-lessons-v3.pdf | Assistant-written summary of Google security publications, October 3, 2026 | secondary summary; its source list includes Google pages, blog posts and papers, and two papers by outside authors. For Google's Secure by Design technical report it read the full text from a third-party mirror (elhacker.info), and cites Google's page for the report, which holds the abstract and links to Google's official PDF: https://storage.googleapis.com/gweb-research2023-media/pubtools/7665.pdf. On October 4, 2026 the summary's statements about that report were compared with the official PDF, and the report's content supports them. One attribution discrepancy: the summary places the syzkaller bug counts after its remark on data races, but the report cites them for bug finding outpacing bug fixing. The report does not disagree with the summary. The summary's Android-blog figures were not checked. The file is unchanged | 482943493ee7156c7e286f82beb80b4c0484d0ad86a2f3c6157c25e807c2b209

—

Appendix E. Trust-ledger candidates (point-in-time)

This appendix adds no requirement, control, test or decision. It lists facts read on October 4, 2026 through the owner's GitHub account and public pages. They may change and do not substitute for the Phase 1 trust-assumption decisions, which were recorded after this appendix was written (D22 to D24, with D25). Where this appendix differs from them, AUDIT.md and the decision files govern. A candidate is an entry Phase 1 may consider for the O6 ledger; it is not a recorded assumption. Repository settings are not inferred from repository files.

Candidate entries:

* GitHub Pages hosting (site soliskit.github.io/tetris-js, source GitHub Actions, HTTPS enforced). Responsibility: serves the built files. GitHub Docs for github.com document no setting for custom Pages response headers; that is an absence of documentation and not proof that it is impossible. A Permissions-Policy header announced in 2021 (https://github.blog/changelog/2021-04-27-github-pages-permissions-policy-interest-cohort-header-

added-to-all-pages-sites/) was not observed on October 4, 2026, through one proxy. Whether a header-setting proxy would work, and whether the lack of frame-ancestors is acceptable, are unverified and are separate owner decisions.
* Browser storage (localStorage). Responsibility: holds saved games. Failure modes: the user or a script on the same origin can change it; Safari may delete it after 7 days of no user interaction unless the game is a home screen web application; whether Safari honors a persistent-storage request is unverified.
* Service worker (public/sw.js). Responsibility: caches versions of the game. Failure modes are the attacks recorded for Phase 4 in Section 4.8.
* GitHub Actions and repository settings. On: dependency graph, Dependabot alerts (0 open), Dependabot security updates, secret scanning alerts, Copilot Autofix. Push protection appears enabled, inferred from a button label, and is unverified. Not set up: CodeQL, Dependabot version updates, dependabot.yml, private vulnerability reporting, SECURITY.md. Actions are referenced by tag, SHA pinning is not required, and the default token is read-only.

Non-goals of this audit's trust accounting: CSP, a header proxy, CodeQL, Dependabot version updates, SHA pinning and storage persistence are not requirements and are not adopted here; each is a separate owner decision. CSRF, CORS and authentication are conditional on a real backend, an authenticated surface or cross-origin use, none of which this project has today. Apple material is not a guarantee for this project. A checksum detects accidental corruption and is not protection against a local player or a malicious change.

Status-label publication migration

Human-authored status labels use title case, with short prepositions such as by and of lowercase. Historical published status records have been restyled as well; status meanings and assignments are unchanged. Original source, review and model-input identities remain historical. AUDIT.md and docs/status-label-migration.md identify the successor publication bytes. Verbatim requirement quotations, runtime identifiers and raw external result strings retain their original spelling.